

Aula 05 - Prática: Desafio Integrativo "DevTech DevOps"

Descrição: Desafio prático focado nas rotinas essenciais de Git e GitHub, estruturado em dinâmica de Dupla Executora e Auditoria Cruzada. Envolve desde a configuração local, gestão de erros e colaboração via Fork/PR até o alinhamento de histórico com Rebase. Capacita os estudantes na resolução negociada de conflitos e na validação técnica por pares com laudo final.

Estrutura da Dinâmica: Dupla Executora e Auditoria Cruzada

A turma será dividida em pares de duplas:

1. **Dupla Executora (Dupla 1 - Alunos A e B):** Realiza toda a implementação do software e operações de Git/GitHub em suas estações.
2. **Dupla Auditora (Dupla 2 - Alunos C e D):** Acompanha a execução, inspeciona o repositório remoto, valida o histórico via terminal/GitHub e preenche o **Laudo de Auditoria Técnica** que será enviado diretamente ao professor.
3. **Inversão:** Na segunda parte da aula ou em ciclo espelhado, a Dupla 2 executa o desafio enquanto a Dupla 1 realiza a auditoria.

Roteiro Prático

FASE 1: Configuração, Identidade e Estruturação Inicial

Objetivos: Identificação de autoria, inicialização de repositório, configuração do .gitignore e registro de tag inicial.

1. **Configuração Global (Alunos A e B):** Ambos devem configurar a identidade do Git no terminal do laboratório:

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu.email@exemplo.com"
```

2. **Ativação do Modo de Inspeção Avançada (Alunos A e B):** Ativem a visualização de três vias para tratamento de conflitos:

```
git config merge.conflictstyle diff3
```

3. **Inicialização Local (Aluno A):**

- o Crie a pasta devtech-sistema e inicialize o repositório:

```
mkdir devtech-sistema
cd devtech-sistema
git init
```

- o Crie o arquivo .gitignore para blindar o projeto contra arquivos sensíveis e de log:

```
# Regras de segurança do projeto
```

```
config_secreta.env  
*.log
```

- Crie o arquivo index.html com a estrutura inicial do sistema.
- Transite os arquivos pelas **3 áreas** (*Working Directory* -> *Staging Area* -> *Local Repository*):

```
git status  
git add .  
git commit -m "feat: estrutura inicial do projeto e configuracao do gitignore"
```

- Marque a versão inicial com uma **Tag Anotada**:

```
git tag -a v0.1.0 -m "Versao de base do sistema"
```

FASE 2: Gestão de Erros, Emergências e Limpeza de Disco

Objetivos: Aplicar técnicas de reversão de erros (restore, amend), isolamento temporário (stash) e remoção de arquivos soltos (clean).

1. Simulação de Erros e Reversão no Working Directory e Stage (Aluno A):

- *Erro no Working Directory:* Adicione uma linha errada no index.html. Restaure o arquivo ao estado do último commit:

```
git restore index.html
```

- *Erro no Stage:* Adicione uma alteração ao stage (git add index.html) e remova-a da área de preparação sem perder o arquivo:

```
git restore --staged index.html
```

- *Erro na Mensagem de Commit:* Faça um commit com mensagem informal e corrija-a com o amend:

```
git commit -m "mudei umas coisas"  
git commit --amend -m "refactor: ajusta identacao do cabecalho principal"
```

2. Atendimento a Emergência com Stash e Clean (Aluno A):

- Inicie a escrita de uma nova função em um arquivo servicos.html (deixe o texto incompleto).
- Surge uma emergência em produção! Guarde o trabalho inacabado no "armário temporário":

```
git stash
```

- Um arquivo malicioso não rastreado chamado temp_hacker.log surge no diretório. Execute a simulação de limpeza e depois force a exclusão:

```
git clean -n  
git clean -f
```

- Com o diretório limpo, altere o index.html corrigindo um erro de digitação, faça o commit e, em seguida, resgate a tarefa que estava guardada no Stash:

```
git commit -am "fix: corrige erro ortografico no rodape"  
git stash pop
```

- Finalize e faça o commit do arquivo servicos.html.
-

FASE 3: Publicação Remota, Fork, Branching e Pull Request

Objetivos: Publicação no GitHub, colaboração via Fork/PR, revisão de código (*Code Review*) e fluxo de trabalho protegido.

1. Publicação no GitHub (Aluno A):

- Crie um repositório público chamado devtech-sistema no GitHub.
- Conecte o repositório local ao remoto e faça o envio da main e das tags:

```
git remote add origin <URL_DO_REPOSITORIO_ALUNO_A>  
git branch -M main  
git push -u origin main  
git push origin --tags
```

2. Contribuição Protegida via Fork (Aluno B):

- Acesse o GitHub do Aluno A e realize um **Fork** do repositório para sua conta pessoal.
- Clone o seu Fork localmente:

```
git clone <URL_DO_FORK_ALUNO_B>  
cd devtech-sistema
```

- Crie uma branch para desenvolver o módulo de contatos:

```
git checkout -b feature/modulo-contato
```

- Crie o arquivo contato.html, adicione o formulário de atendimento, faça o commit e envie para o seu Fork remoto:

```
git add contato.html  
git commit -m "feat: implementa formulario na pagina de contato"  
git push origin feature/modulo-contato
```

- No GitHub, abra um **Pull Request (PR)** da sua branch feature/modulo-contato apontando para a branch main do repositório original do Aluno A.

3. Code Review e Merge do PR (Aluno A):

- O Aluno A acessa a aba *Pull Requests*, analisa as alterações enviadas pelo Aluno B, escreve um comentário de aprovação e executa o **Merge** no GitHub.
-

FASE 4: Rebase, Desenvolvimento Divergente e Resolução de Conflitos

Objetivos: Organização do histórico com rebase, geração proposital de conflito, negociação técnica em dupla, limpeza de marcadores e lançamento com tag.

1. Alinhamento com Rebase (Aluno B):

- Aluno B atualiza sua branch main com as mudanças aprovadas do repositório original e alinha sua histórico utilizando o Rebase para manter uma linha do tempo linear:

```
git checkout main
git pull origin main
git rebase main feature/modulo-contato
```

2. Simulação da Colisão de Códigos (Alunos A e B):

- Ambos os alunos sincronizam suas branches main locais (git pull origin main).
- **Aluno A:** Crie a branch feature/banner-azul, edite a **linha 10** do arquivo index.html inserindo `<header class="bg-azul">DevTech Azul</header>`, faça o commit e mescle na main no GitHub.
- **Aluno B (Simultaneamente):** Crie a branch feature/banner-verde, edite a **MESMA linha 10** do arquivo index.html inserindo `<header class="bg-verde">DevTech Verde</header>` e faça o commit local.

3. O Conflito e a Negociação (Trabalho Presencial da Dupla):

- Aluno B tenta puxar a main atualizada e realizar a fusão local:

```
git checkout main
git pull origin main
git merge feature/banner-verde
```

- O Git interromperá o processo informando **CONFLICT (content)**.
- **Ação em Dupla:** Ambos analisam o código no VS Code. Por conta da configuração diff3, o arquivo exibirá o ancestral comum (|||||), a versão atual (<<<<<<) e a versão recebida (>>>>>>).
- Os alunos negociam uma solução unificada (ex: criar uma estrutura reutilizável com ambas as opções estilizadas).
- **Limpeza:** Removem manualmente todos os marcadores (<<<<<<, |||||, =====, >>>>>>).
- Concluem a fusão no terminal:

```
git add index.html
```

```
git commit -m "fix: resolve conflito integrando os temas de cabeçalho"  
git push origin main
```

- Criam a tag oficial da entrega e enviam ao GitHub:

```
git tag -a v1.0.0 -m "Versao 1.0.0 finalizada e auditada"  
git push origin --tags
```

FASE 5: Processo de Auditoria Cruzada e Envio

Objetivos: Validação por pares, checagem do repositório remoto, terminal e geração do laudo de entrega.

1. A **Dupla Auditora** assume a máquina da Dupla Executora e acessa a página do GitHub do projeto.
2. A Dupla Auditora executa os comandos de inspeção no terminal:

```
git log --oneline --graph --all  
git tag  
git status
```

3. Preenchem o laudo e enviam o arquivo consolidado para o professor via Ambiente Virtual de Aprendizagem (AVA).
4. Lista de Entregáveis Obrigatórios. Para a conclusão da atividade com êxito, a equipe e a dupla auditora devem entregar:
 - A. **Link do Repositório Remoto no GitHub:** Contendo o projeto completo com histórico de commits organizado.
 - B. **Histórico Rastreável de Branches e Merges:** Comprovação da criação de branches (feature/*), fusões e histórico mantido sem commits corrompidos.
 - C. **Registro do Pull Request (PR):** Histórico no GitHub demonstrando a contribuição via Fork, revisão de código e mesclagem.
 - D. **Resolução de Conflito Válida:** Código-fonte 100% limpo, sem nenhum marcador de conflito remanescente (<<<<<<, =====, >>>>>>) e sem erros de sintaxe.
 - E. **Tags de Versão Registrar:** Tags v0.1.0 e v1.0.0 corretamente anotadas e sincronizadas no GitHub.
 - F. **Laudo de Auditoria Técnica:** Documento preenchido pela dupla auditora e assinado pelos integrantes, contendo o parecer final da entrega.

5. Laudo de Auditoria Técnica (Modelo de Enviado pelo Professor no AVA)
-